```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

TMDB Movies Dataset Description

This dataset contains information about movies from The Movie Database (TMDB). It includes details such as movie titles, release dates, budgets, revenues, ratings, and more.

Main Columns:

1. **id**: Unique identifier for the movie.
2. **imdb_id**: Unique identifier for the movie in IMDb.
3. **popularity**: A numeric value representing the movie's popularity.
4. **budget**: The budget of the movie (in USD).
5. **revenue**: The revenue generated by the movie (in USD).
6. **original_title**: The original title of the movie.
7. **cast**: A list of actors/actresses in the movie.
8. **homepage**: The official homepage of the movie (if available).
9. **director**: The director of the movie.
10. **tagline**: A short tagline or slogan for the movie.
11. **keywords**: Keywords or tags associated with the movie.
12. **overview**: A brief summary or description of the movie.
13. **runtime**: The duration of the movie (in minutes).
14. **genres**: A list of genres the movie belongs to (e.g., Action, Drama, Comedy).
15. **production_companies**: A list of companies involved in the movie's production.
16. **release_date**: The release date of the movie.
17. **vote_count**: The number of votes the movie received.
18. **vote_average**: The average rating of the movie (out of 10).
19. **release_year**: The year the movie was released (extracted from release_date).

Key Insights and ### Example Questions:

This dataset can be used to analyze as:

- Which genres are the most popular from year to year?
- What kinds of properties are associated with movies that have high revenues?
- Which directors have the highest average movie ratings?
- Who are the Top 3 Directors according their revenues
- What is The average revenue per genre
- What are the Highest-rated movies.
- Trends in movie releases over time.

- Count the number of movies per genre
- High revenue movies VS Other Movies

Initial Data Exploration

```
df= pd.read_csv('tmdb-movies.csv')
# read the first 5 rows in the data
df.head(5)
```

```
       id    imdb_id  popularity     budget     revenue  \
0  135397  tt0369610   32.985763  150000000  1513528810
1   76341  tt1392190   28.419936  150000000   378436354
2  262500  tt2908446   13.112507  110000000   295238201
3  140607  tt2488496   11.173104  200000000  2068178225
4  168259  tt2820852    9.335014  190000000  1506249360

                 original_title  \
0               Jurassic World
1           Mad Max: Fury Road
2                     Insurgent
3  Star Wars: The Force Awakens
4                     Furious 7

                                                cast  \
0  Chris Pratt|Bryce Dallas Howard|Irrfan Khan|Vi...
1  Tom Hardy|Charlize Theron|Hugh Keays-Byrne|Nic...
2  Shailene Woodley|Theo James|Kate Winslet|Ansel...
3  Harrison Ford|Mark Hamill|Carrie Fisher|Adam D...
4  Vin Diesel|Paul Walker|Jason Statham|Michelle ...

                                           homepage           director
\
0                     http://www.jurassicworld.com/   Colin Trevorrow

1                       http://www.madmaxmovie.com/     George Miller

2    http://www.thedivergentseries.movie/#insurgent  Robert Schwentke

3  http://www.starwars.com/films/star-wars-episod...       J.J. Abrams

4                           http://www.furious7.com/         James Wan


                        tagline  ...  \
0             The park is open.  ...
1            What a Lovely Day.  ...
2    One Choice Can Destroy You  ...
3  Every generation has a story.  ...
4           Vengeance Hits Home  ...
```

```
                                                overview runtime  \
0  Twenty-two years after the events of Jurassic ...     124
1  An apocalyptic story set in the furthest reach...     120
2  Beatrice Prior must confront her inner demons ...     119
3  Thirty years after defeating the Galactic Empi...     136
4  Deckard Shaw seeks revenge against Dominic Tor...     137

                                       genres  \
0  Action|Adventure|Science Fiction|Thriller
1  Action|Adventure|Science Fiction|Thriller
2         Adventure|Science Fiction|Thriller
3   Action|Adventure|Science Fiction|Fantasy
4                     Action|Crime|Thriller

                                  production_companies release_date
vote_count  \
0  Universal Studios|Amblin Entertainment|Legenda...       6/9/15
5562
1  Village Roadshow Pictures|Kennedy Miller Produ...      5/13/15
6185
2  Summit Entertainment|Mandeville Films|Red Wago...      3/18/15
2480
3          Lucasfilm|Truenorth Productions|Bad Robot     12/15/15
5292
4  Universal Pictures|Original Film|Media Rights ...       4/1/15
2947

   vote_average  release_year    budget_adj   revenue_adj
0           6.5          2015  1.379999e+08  1.392446e+09
1           7.1          2015  1.379999e+08  3.481613e+08
2           6.3          2015  1.012000e+08  2.716190e+08
3           7.5          2015  1.839999e+08  1.902723e+09
4           7.3          2015  1.747999e+08  1.385749e+09

[5 rows x 21 columns]
```

```
# read the least 5 rows in the data
df.tail(5)
```

```
          id    imdb_id  popularity  budget  revenue  \
10861     21  tt0060371    0.080598       0        0
10862  20379  tt0060472    0.065543       0        0
10863  39768  tt0060161    0.065141       0        0
10864  21449  tt0061177    0.064317       0        0
10865  22293  tt0060666    0.035919   19000        0

                original_title  \
10861        The Endless Summer
10862                Grand Prix
10863       Beregis Avtomobilya
```

```
10864    What's Up, Tiger Lily?
10865  Manos: The Hands of Fate

                                                    cast homepage  \
10861  Michael Hynson|Robert August|Lord 'Tally Ho' B...      NaN
10862  James Garner|Eva Marie Saint|Yves Montand|Tosh...      NaN
10863  Innokentiy Smoktunovskiy|Oleg Efremov|Georgi Z...      NaN
10864  Tatsuya Mihashi|Akiko Wakabayashi|Mie Hama|Joh...      NaN
10865  Harold P. Warren|Tom Neyman|John Reynolds|Dian...      NaN

                director
tagline  \
10861         Bruce Brown
NaN
10862  John Frankenheimer  Cinerama sweeps YOU into a drama of speed
and ...
10863      Eldar Ryazanov
NaN
10864         Woody Allen                             WOODY ALLEN STRIKES
BACK!
10865    Harold P. Warren       It's Shocking! It's Beyond Your
Imagination!

       ...                                              overview runtime
\
10861  ...  The Endless Summer, by Bruce Brown, is one of ...      95

10862  ...  Grand Prix driver Pete Aron is fired by his te...     176

10863  ...  An insurance agent who moonlights as a carthie...      94

10864  ...  In comic Woody Allen's film debut, he took the...      80

10865  ...  A family gets lost on the road and stumbles up...      74


                      genres  \
10861             Documentary
10862  Action|Adventure|Drama
10863         Mystery|Comedy
10864          Action|Comedy
10865                 Horror

                                    production_companies release_date
\
10861                                  Bruce Brown Films      6/15/66

10862  Cherokee Productions|Joel Productions|Douglas ...     12/21/66

10863                                          Mosfilm       1/1/66
```

```
10864                                Benedict Pictures Corp.      11/2/66

10865                                                Norm-Iris     11/15/66


      vote_count  vote_average  release_year     budget_adj
revenue_adj
10861         11           7.4          1966       0.000000
0.0
10862         20           5.7          1966       0.000000
0.0
10863         11           6.5          1966       0.000000
0.0
10864         22           5.4          1966       0.000000
0.0
10865         15           1.5          1966  127642.279154
0.0

[5 rows x 21 columns]
```

```
# Basic statistics about the dataset:
print("\nBasic statistics about the dataset:")
df.describe()
```

```
Basic statistics about the dataset:

                  id    popularity        budget       revenue
runtime  \
count   10866.000000  10866.000000  1.086600e+04  1.086600e+04
10866.000000
mean    66064.177434      0.646441  1.462570e+07  3.982332e+07
102.070863
std     92130.136561      1.000185  3.091321e+07  1.170035e+08
31.381405
min         5.000000      0.000065  0.000000e+00  0.000000e+00
0.000000
25%     10596.250000      0.207583  0.000000e+00  0.000000e+00
90.000000
50%     20669.000000      0.383856  0.000000e+00  0.000000e+00
99.000000
75%     75610.000000      0.713817  1.500000e+07  2.400000e+07
111.000000
max    417859.000000     32.985763  4.250000e+08  2.781506e+09
900.000000

         vote_count  vote_average  release_year    budget_adj
revenue_adj
count  10866.000000  10866.000000  10866.000000  1.086600e+04
1.086600e+04
```

```
mean     217.389748      5.974922   2001.322658  1.755104e+07
5.136436e+07
std      575.619058      0.935142     12.812941  3.430616e+07
1.446325e+08
min       10.000000      1.500000   1960.000000  0.000000e+00
0.000000e+00
25%       17.000000      5.400000   1995.000000  0.000000e+00
0.000000e+00
50%       38.000000      6.000000   2006.000000  0.000000e+00
0.000000e+00
75%      145.750000      6.600000   2011.000000  2.085325e+07
3.369710e+07
max     9767.000000      9.200000   2015.000000  4.250000e+08
2.827124e+09
```

```
print("\nSize of the dataset (rows, columns):")
df.shape
```

```
Size of the dataset (rows, columns):

(10866, 21)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10866 entries, 0 to 10865
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   id                    10866 non-null  int64
 1   imdb_id               10856 non-null  object
 2   popularity            10866 non-null  float64
 3   budget                10866 non-null  int64
 4   revenue               10866 non-null  int64
 5   original_title        10866 non-null  object
 6   cast                  10790 non-null  object
 7   homepage              2936 non-null   object
 8   director              10822 non-null  object
 9   tagline               8042 non-null   object
 10  keywords              9373 non-null   object
 11  overview              10862 non-null  object
 12  runtime               10866 non-null  int64
 13  genres                10843 non-null  object
 14  production_companies  9836 non-null   object
 15  release_date          10866 non-null  object
 16  vote_count            10866 non-null  int64
 17  vote_average          10866 non-null  float64
 18  release_year          10866 non-null  int64
 19  budget_adj            10866 non-null  float64
```

```
 20  revenue_adj           10866 non-null  float64
dtypes: float64(4), int64(6), object(11)
memory usage: 1.7+ MB
```

```
# Missing values
print("\nMissing values in each column:")
df.isnull().sum()
```

```
Missing values in each column:

id                         0
imdb_id                   10
popularity                 0
budget                     0
revenue                    0
original_title             0
cast                      76
homepage                7930
director                  44
tagline                 2824
keywords                1493
overview                   4
runtime                    0
genres                    23
production_companies    1030
release_date               0
vote_count                 0
vote_average               0
release_year               0
budget_adj                 0
revenue_adj                0
dtype: int64
```

Clean the Data

Drop unnecessary columns :(

```
df.drop(['homepage', 'tagline', 'keywords', 'overview',
'imdb_id','production_companies'], axis = 1, inplace = True)

df.head()
```

```
       id  popularity     budget     revenue
original_title  \
0  135397   32.985763  150000000  1513528810                Jurassic
World
1   76341   28.419936  150000000   378436354            Mad Max: Fury
Road
2  262500   13.112507  110000000   295238201
```

```
Insurgent
3  140607   11.173104  200000000  2068178225  Star Wars: The Force
Awakens
4  168259    9.335014  190000000  1506249360
Furious 7

                                                cast          director
\
0  Chris Pratt|Bryce Dallas Howard|Irrfan Khan|Vi...   Colin Trevorrow

1  Tom Hardy|Charlize Theron|Hugh Keays-Byrne|Nic...     George Miller

2  Shailene Woodley|Theo James|Kate Winslet|Ansel...  Robert Schwentke

3  Harrison Ford|Mark Hamill|Carrie Fisher|Adam D...       J.J. Abrams

4  Vin Diesel|Paul Walker|Jason Statham|Michelle ...          James Wan


   runtime                                     genres release_date  \
0      124  Action|Adventure|Science Fiction|Thriller       6/9/15
1      120  Action|Adventure|Science Fiction|Thriller      5/13/15
2      119         Adventure|Science Fiction|Thriller      3/18/15
3      136   Action|Adventure|Science Fiction|Fantasy     12/15/15
4      137                      Action|Crime|Thriller       4/1/15

   vote_count  vote_average  release_year    budget_adj   revenue_adj

0        5562           6.5          2015  1.379999e+08  1.392446e+09

1        6185           7.1          2015  1.379999e+08  3.481613e+08

2        2480           6.3          2015  1.012000e+08  2.716190e+08

3        5292           7.5          2015  1.839999e+08  1.902723e+09

4        2947           7.3          2015  1.747999e+08  1.385749e+09
```

Fill missing values in (vote_average) with the mean

```
# As 'vote_average' is numerical column so... fill the null values
withe the mean :)
df['vote_average'].fillna(df['vote_average'].mean(), inplace=True)

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10866 entries, 0 to 10865
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
```

```
---  ------          --------------  -----
 0   id              10866 non-null  int64
 1   popularity      10866 non-null  float64
 2   budget          10866 non-null  int64
 3   revenue         10866 non-null  int64
 4   original_title  10866 non-null  object
 5   cast            10790 non-null  object
 6   director        10822 non-null  object
 7   runtime         10866 non-null  int64
 8   genres          10843 non-null  object
 9   release_date    10866 non-null  object
 10  vote_count      10866 non-null  int64
 11  vote_average    10866 non-null  float64
 12  release_year    10866 non-null  int64
 13  budget_adj      10866 non-null  float64
 14  revenue_adj     10866 non-null  float64
dtypes: float64(4), int64(6), object(5)
memory usage: 1.2+ MB
```

Drop rows with missing values (important columns) :)

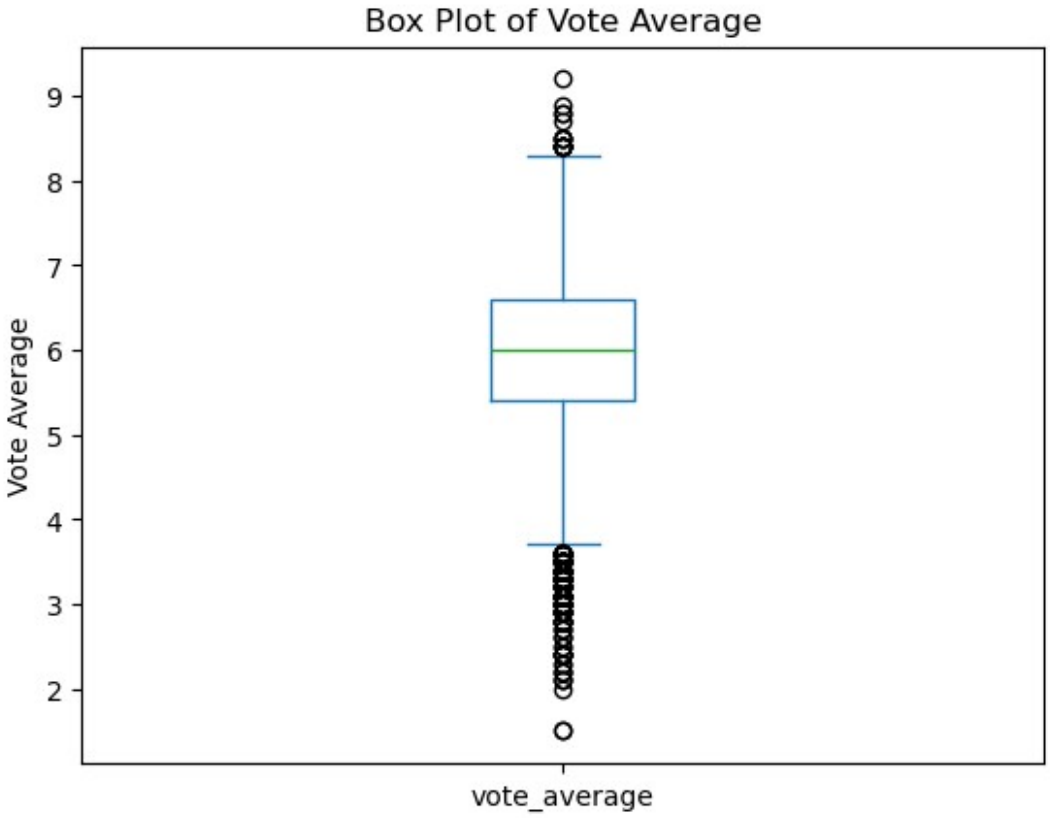
```
df = df.dropna(subset=['genres', 'revenue','director'])

df.info()

<class 'pandas.core.frame.DataFrame'>
Index: 10801 entries, 0 to 10865
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id              10801 non-null  int64
 1   popularity      10801 non-null  float64
 2   budget          10801 non-null  int64
 3   revenue         10801 non-null  int64
 4   original_title  10801 non-null  object
 5   cast            10732 non-null  object
 6   director        10801 non-null  object
 7   runtime         10801 non-null  int64
 8   genres          10801 non-null  object
 9   release_date    10801 non-null  object
 10  vote_count      10801 non-null  int64
 11  vote_average    10801 non-null  float64
 12  release_year    10801 non-null  int64
 13  budget_adj      10801 non-null  float64
 14  revenue_adj     10801 non-null  float64
dtypes: float64(4), int64(6), object(5)
memory usage: 1.3+ MB
```

What is the most popular genery from year to year :) ?

```
df.head()
```

```
       id  popularity     budget     revenue
original_title  \
0  135397   32.985763  150000000  1513528810                Jurassic
World
1   76341   28.419936  150000000   378436354             Mad Max: Fury
Road
2  262500   13.112507  110000000   295238201
Insurgent
3  140607   11.173104  200000000  2068178225  Star Wars: The Force
Awakens
4  168259    9.335014  190000000  1506249360
Furious 7

                                                cast           director
\
0  Chris Pratt|Bryce Dallas Howard|Irrfan Khan|Vi...   Colin Trevorrow

1  Tom Hardy|Charlize Theron|Hugh Keays-Byrne|Nic...     George Miller

2  Shailene Woodley|Theo James|Kate Winslet|Ansel...  Robert Schwentke

3  Harrison Ford|Mark Hamill|Carrie Fisher|Adam D...       J.J. Abrams

4  Vin Diesel|Paul Walker|Jason Statham|Michelle ...          James Wan


   runtime                                      genres release_date  \
0      124  Action|Adventure|Science Fiction|Thriller       6/9/15
1      120  Action|Adventure|Science Fiction|Thriller      5/13/15
2      119         Adventure|Science Fiction|Thriller      3/18/15
3      136   Action|Adventure|Science Fiction|Fantasy     12/15/15
4      137                      Action|Crime|Thriller       4/1/15

   vote_count  vote_average  release_year    budget_adj   revenue_adj

0        5562           6.5          2015  1.379999e+08  1.392446e+09

1        6185           7.1          2015  1.379999e+08  3.481613e+08

2        2480           6.3          2015  1.012000e+08  2.716190e+08

3        5292           7.5          2015  1.839999e+08  1.902723e+09

4        2947           7.3          2015  1.747999e+08  1.385749e+09
```

```
# Making Box Plot for column : vote_average
df['vote_average'].plot(kind='box', title='Box Plot of Vote Average')
plt.ylabel('Vote Average')
plt.show()
```




```
df['genres']
```

```
0        Action|Adventure|Science Fiction|Thriller
1        Action|Adventure|Science Fiction|Thriller
2               Adventure|Science Fiction|Thriller
3         Action|Adventure|Science Fiction|Fantasy
4                           Action|Crime|Thriller
                             ...
10861                                 Documentary
10862                      Action|Adventure|Drama
10863                             Mystery|Comedy
10864                              Action|Comedy
10865                                     Horror
Name: genres, Length: 10801, dtype: object
```

```
# function to find the most popular genres for each release year.
def get_most_popular_genres(df):
    df['genres'] = df['genres'].str.split('|')
    genres_df = df.explode('genres')
```

```
    genres_popularity = genres_df.groupby(['release_year',
'genres']).size().unstack()
    top_popular_genres = genres_popularity.idxmax(axis=1)
    most_popular_genres= get_most_popular_genres(df)
```

The most popular genery from year to year

```
print('\nThe most popular genery from year to year')
pd.DataFrame(most_popular_genres)
```

```
The most popular genery from year to year
```

release_year	0
1960	Drama
1961	Drama
1962	Drama
1963	Comedy
1964	Drama
1965	Drama
1966	Comedy
1967	Comedy
1968	Drama
1969	Drama
1970	Drama
1971	Drama
1972	Drama
1973	Drama
1974	Drama
1975	Drama
1976	Drama
1977	Drama
1978	Drama
1979	Drama
1980	Drama
1981	Drama
1982	Drama
1983	Drama
1984	Drama
1985	Comedy
1986	Drama
1987	Comedy
1988	Comedy
1989	Comedy
1990	Drama
1991	Drama
1992	Drama
1993	Drama
1994	Comedy

```
1995           Drama
1996           Drama
1997           Drama
1998           Drama
1999           Drama
2000           Drama
2001          Comedy
2002           Drama
2003          Comedy
2004           Drama
2005           Drama
2006           Drama
2007           Drama
2008           Drama
2009           Drama
2010           Drama
2011           Drama
2012           Drama
2013           Drama
2014           Drama
2015           Drama
```

Count the number of movies per genre

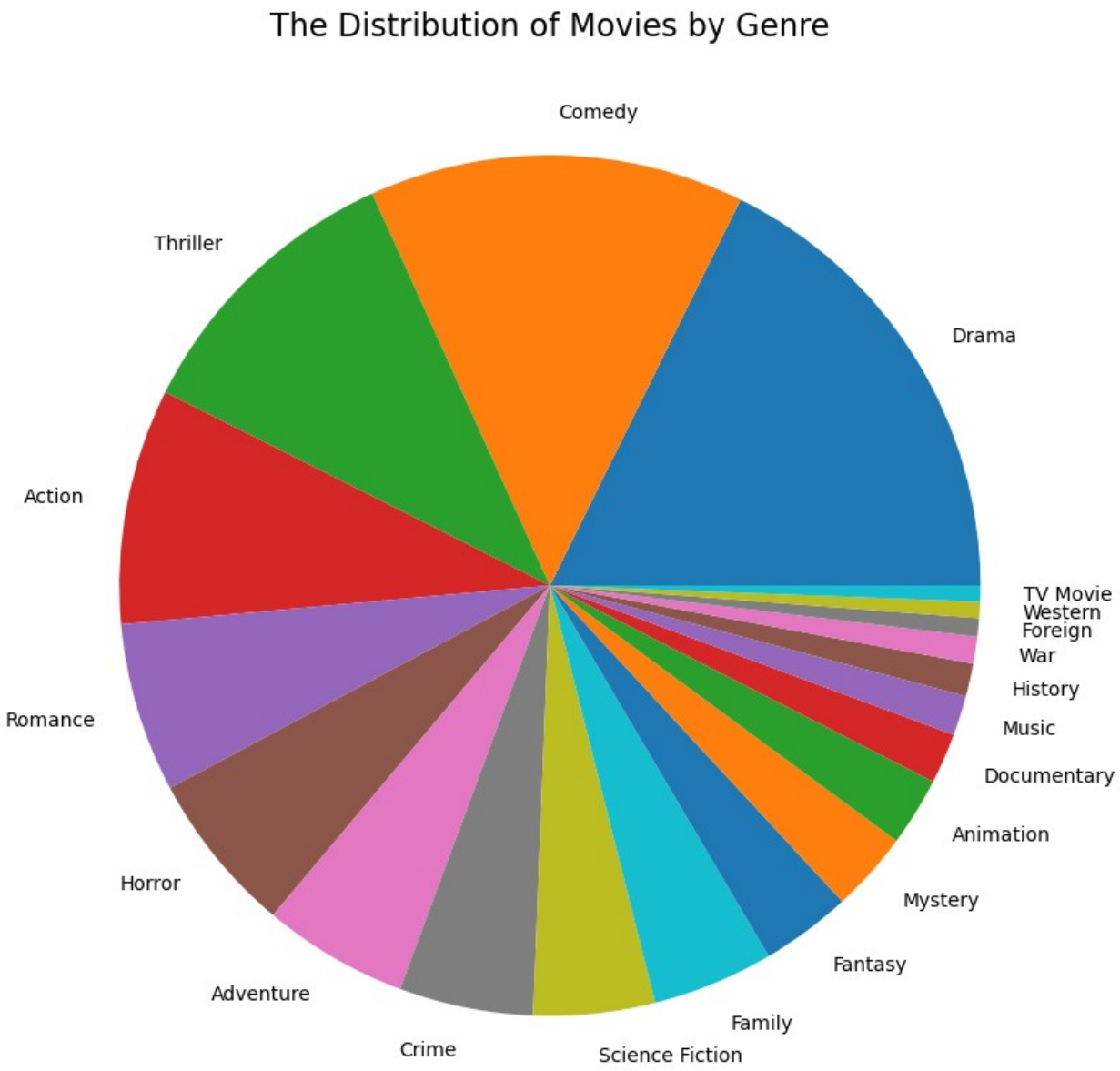
```
# 1d ..............
genre_counts = genres_df['genres'].value_counts()
print("\nThe sum of movies in each genre the : ")
genre_counts
```

```
The sum of movies in each genre the :

genres
Drama               4755
Comedy              3782
Thriller            2905
Action              2379
Romance             1708
Horror              1636
Adventure           1466
Crime               1354
Science Fiction     1224
Family              1223
Fantasy              912
Mystery              809
Animation            692
Documentary          509
Music                402
History              332
War                  270
```

```
Foreign              185
Western              164
TV Movie             162
Name: count, dtype: int64
```

(Drame) has the highst no. of movies by 4761 movie

```
plt.figure(figsize=(15, 10))
plt.pie(genre_counts, labels= genre_counts.index)
plt.title('The Distribution of Movies by Genre', fontsize=16)
plt.show()
```



Calculate the average revenue per genre

```
df.head()
```

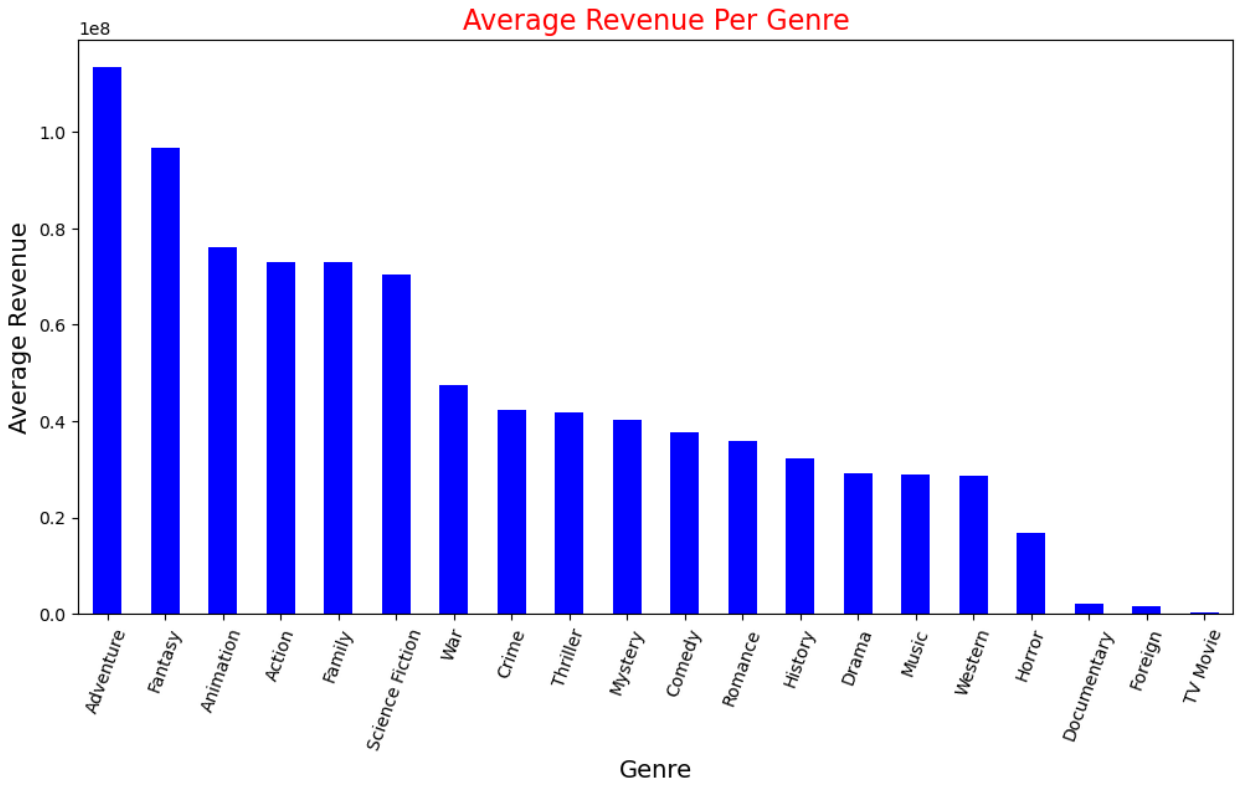
```
       id  popularity     budget     revenue
original_title  \
0  135397   32.985763  150000000  1513528810                Jurassic
World
1   76341   28.419936  150000000   378436354             Mad Max: Fury
Road
2  262500   13.112507  110000000   295238201
Insurgent
3  140607   11.173104  200000000  2068178225  Star Wars: The Force
Awakens
4  168259    9.335014  190000000  1506249360
Furious 7

                                                cast           director
\
0  Chris Pratt|Bryce Dallas Howard|Irrfan Khan|Vi...   Colin Trevorrow

1  Tom Hardy|Charlize Theron|Hugh Keays-Byrne|Nic...     George Miller

2  Shailene Woodley|Theo James|Kate Winslet|Ansel...  Robert Schwentke

3  Harrison Ford|Mark Hamill|Carrie Fisher|Adam D...       J.J. Abrams

4  Vin Diesel|Paul Walker|Jason Statham|Michelle ...          James Wan


   runtime                                      genres release_date  \
0      124  Action|Adventure|Science Fiction|Thriller       6/9/15
1      120  Action|Adventure|Science Fiction|Thriller      5/13/15
2      119         Adventure|Science Fiction|Thriller      3/18/15
3      136   Action|Adventure|Science Fiction|Fantasy     12/15/15
4      137                      Action|Crime|Thriller       4/1/15

   vote_count  vote_average  release_year    budget_adj   revenue_adj

0        5562           6.5          2015  1.379999e+08  1.392446e+09

1        6185           7.1          2015  1.379999e+08  3.481613e+08

2        2480           6.3          2015  1.012000e+08  2.716190e+08

3        5292           7.5          2015  1.839999e+08  1.902723e+09

4        2947           7.3          2015  1.747999e+08  1.385749e+09
```

```
avg_revenue_per_genre = genres_df.groupby('genres')
['revenue'].mean().sort_values(ascending=False)
```

```
pd.DataFrame(avg_revenue_per_genre)
```

genres	revenue
Adventure	1.135237e+08
Fantasy	9.673609e+07
Animation	7.601732e+07
Action	7.294813e+07
Family	7.290698e+07
Science Fiction	7.042787e+07
War	4.760518e+07
Crime	4.236938e+07
Thriller	4.175748e+07
Mystery	4.026728e+07
Comedy	3.763248e+07
Romance	3.576912e+07
History	3.220464e+07
Drama	2.926088e+07
Music	2.899821e+07
Western	2.874291e+07
Horror	1.683309e+07
Documentary	2.085217e+06
Foreign	1.485656e+06
TV Movie	2.592593e+05

```
avg_revenue_per_genre.plot(kind='bar', figsize=(12, 6), color='blue')
plt.title('Average Revenue Per Genre', fontsize=16, color='red')
plt.xlabel('Genre', fontsize=14)
plt.ylabel('Average Revenue', fontsize=14)
plt.xticks(rotation=70)
plt.show()
```

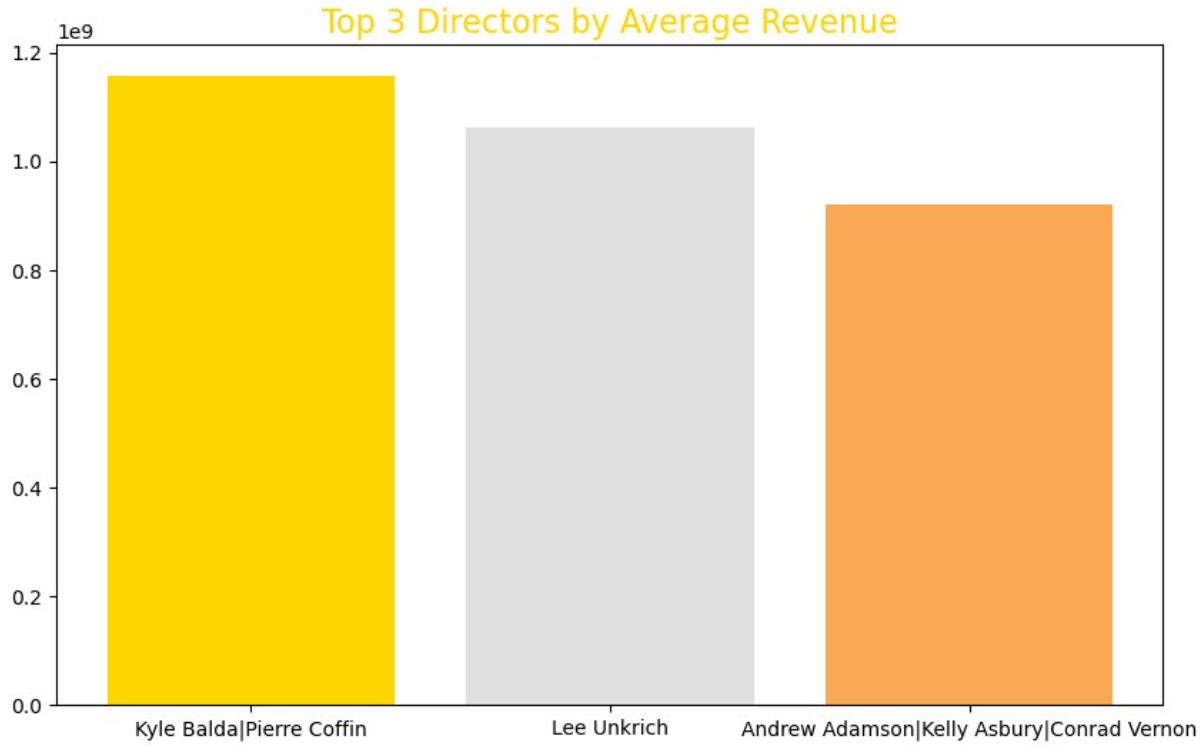


Top 3 Directors according their revenues :)

```
avg_revenue_per_director= df.groupby('director')
['revenue'].mean().sort_values(ascending=False).head(3)

pd.DataFrame(avg_revenue_per_director)
```

director	revenue
Kyle Balda|Pierre Coffin	1.156731e+09
Lee Unkrich	1.063172e+09
Andrew Adamson|Kelly Asbury|Conrad Vernon	9.198388e+08

```
plt.figure(figsize=(10, 6))
plt.bar(avg_revenue_per_director.index,
avg_revenue_per_director.values, color=['#FFD700', '#e0e0e0',
'#fca956'])
plt.title('Top 3 Directors by Average Revenue',
fontsize=16,color='#FFD700')
plt.xlabel('Director', fontsize=14, color='white')
plt.ylabel('Average Revenue', fontsize=14, color='white')
plt.show()
```



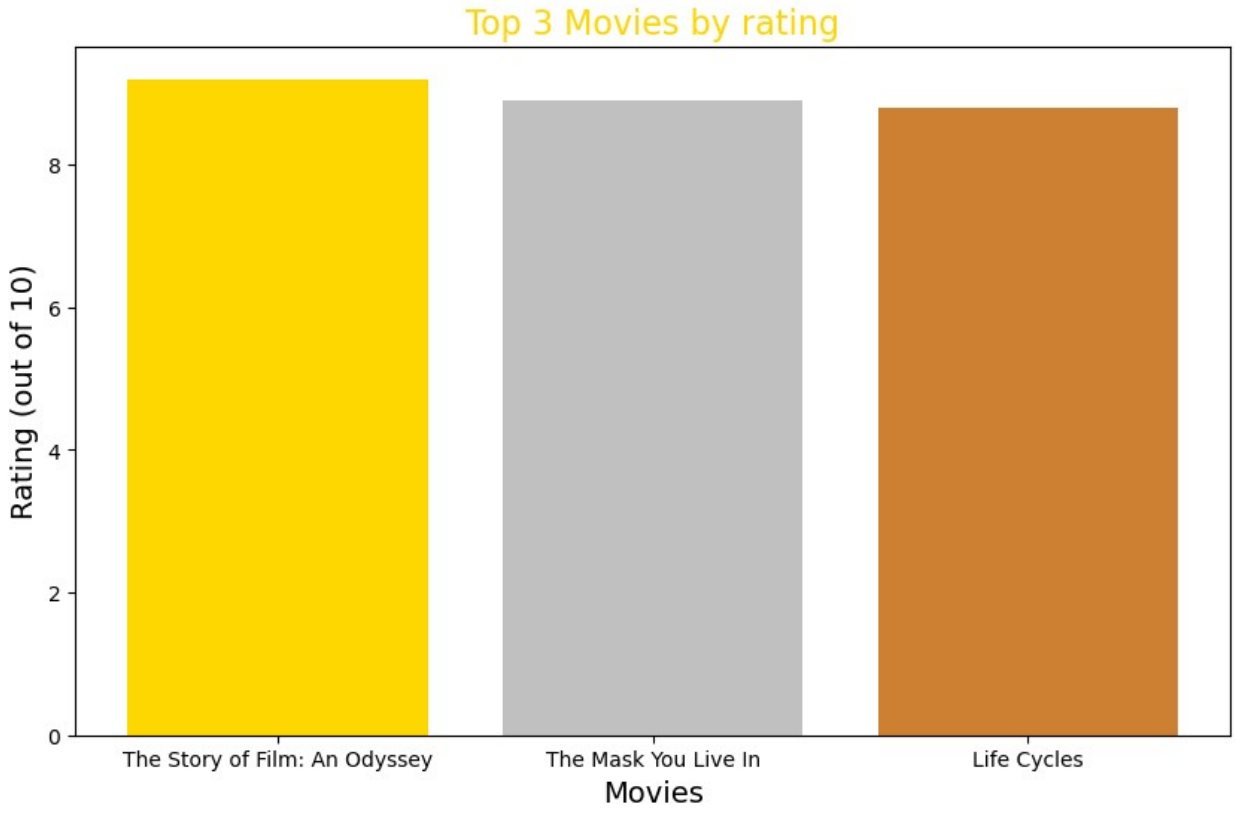
Highest-rated movies.

```
print("\nTop 3 movies by rating:")
# make var. top_3_ratings that contain (top 3) ... of  df:
('original_title', 'release_year', 'vote_average') and sorted
according to 'vote_average'
top_3_ratings = df[['original_title', 'release_year',
'vote_average']].sort_values(by='vote_average',
ascending=False).head(3)
pd.DataFrame(top_3_ratings)
```

```
Top 3 movies by rating:
```

	original_title	release_year	vote_average
3894	The Story of Film: An Odyssey	2011	9.2
538	The Mask You Live In	2015	8.9
2269	Life Cycles	2010	8.8

```
plt.figure(figsize=(10, 6))
plt.bar(top_3_ratings['original_title'],
top_3_ratings['vote_average'], color=['#FFD700', '#C0C0C0',
'#CD7F32'])
plt.title('Top 3 Movies by rating', fontsize=16, color='#FFD700')
plt.xlabel('Movies', fontsize=14, color='black')
plt.ylabel('Rating (out of 10)', fontsize=14, color='black')
plt.show()
```




```
# Define high-revenue movies (top 5%)
revenue_comp = np.percentile(df['revenue'], 95)  # 95th percentile by
(numpy)
high_revenue_movies = df[df['revenue'] > revenue_comp]
other_movies = df[df['revenue'] <= revenue_comp]
```

Compare properties of high-revenue movies vs others

```
print("Properties of High-Revenue Movies:")
print(high_revenue_movies[['budget', 'vote_average', 'popularity',
'runtime']].describe())

print("\nProperties of Other Movies:")
print(other_movies[['budget', 'vote_average', 'popularity',
'runtime']].describe())
```

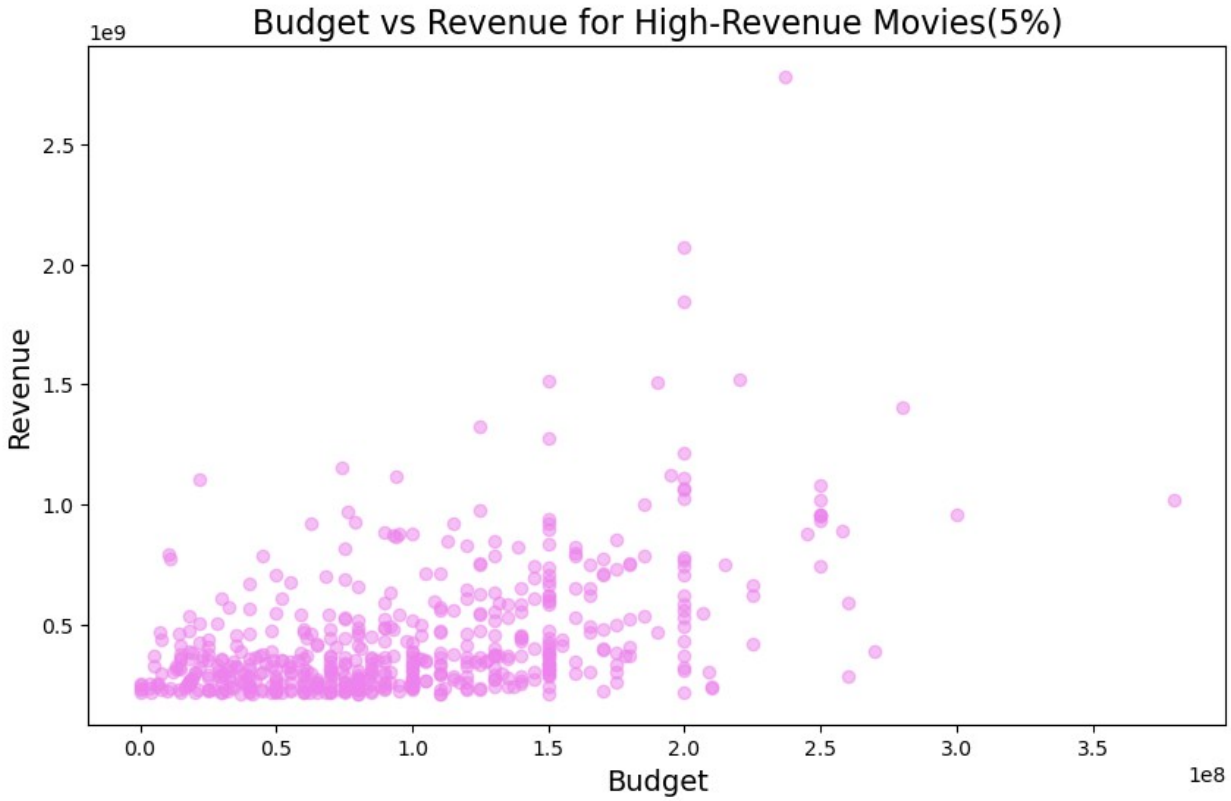
```
Properties of High-Revenue Movies:
             budget  vote_average  popularity     runtime
count  5.400000e+02    540.000000  540.000000  540.000000
mean   9.800699e+07      6.512222    2.914378  117.885185
std    6.154738e+07      0.709123    2.829072   22.256011
min    0.000000e+00      4.400000    0.131526   79.000000
25%    5.000000e+07      6.000000    1.445801  100.000000
50%    9.000000e+07      6.500000    2.199646  116.000000
75%    1.400000e+08      7.000000    3.461242  131.000000
```

```
max    3.800000e+08      8.300000   32.985763  201.000000

Properties of Other Movies:
             budget  vote_average    popularity       runtime
count  1.026100e+04  10261.000000  10261.000000  10261.000000
mean   1.032906e+07      5.942296      0.530255    101.362733
std    2.067047e+07      0.934701      0.594148     30.947849
min    0.000000e+00      1.500000      0.000188      0.000000
25%    0.000000e+00      5.400000      0.202017     90.000000
50%    0.000000e+00      6.000000      0.364854     98.000000
75%    1.200000e+07      6.600000      0.642207    110.000000
max    4.250000e+08      9.200000     11.422751    900.000000
```

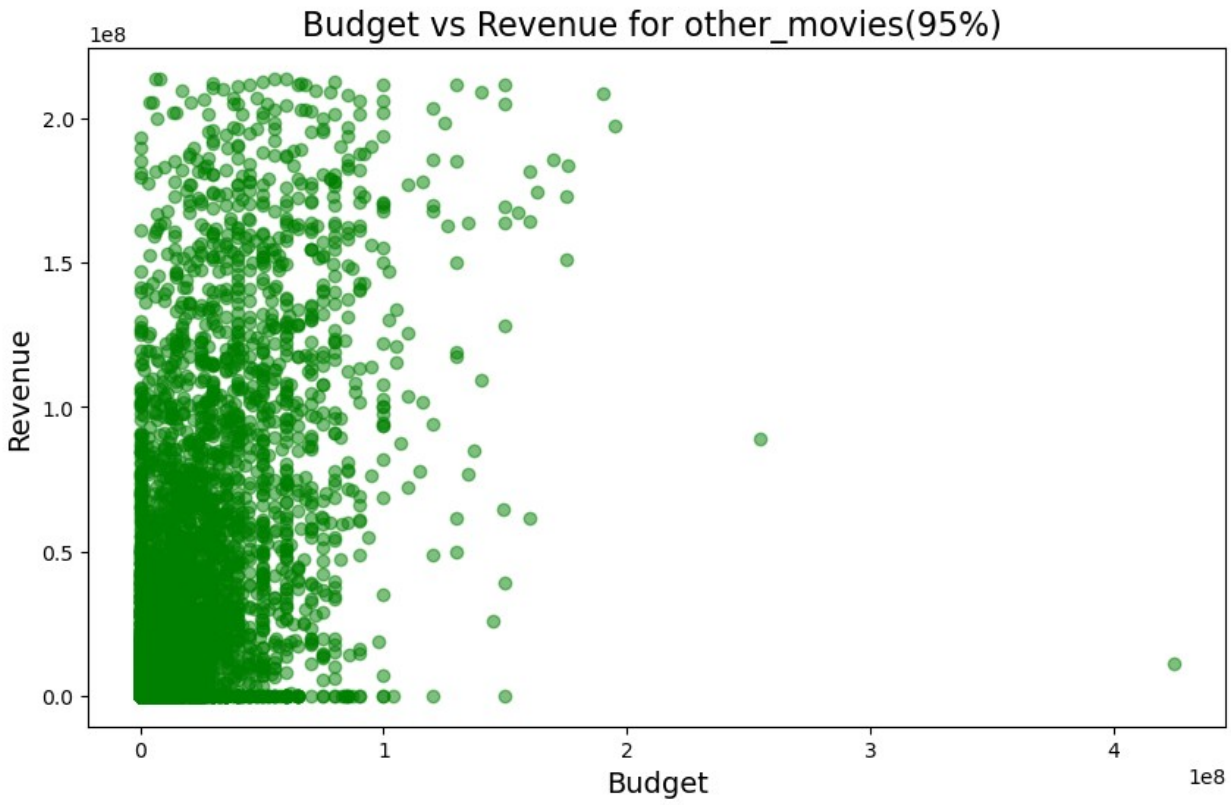
high revenue movies VS Other Movies

- that high revenue movies have significantly higher budgets.
- High high revenue movies have higher average ratings.
- High high revenue movies are more popular.
- High high revenue movies are slightly longer in duration.

```
# Plot the relationship between budget and revenue
plt.figure(figsize=(10, 6))
plt.scatter(high_revenue_movies['budget'],
high_revenue_movies['revenue'], alpha=0.5, color='violet')
plt.title('Budget vs Revenue for High-Revenue Movies(5%)',
fontsize=16)
plt.xlabel('Budget', fontsize=14)
plt.ylabel('Revenue', fontsize=14)
plt.show()
```




```
# Plot the relationship between budget and revenue
plt.figure(figsize=(10, 6))
plt.scatter(other_movies['budget'], other_movies['revenue'],
alpha=0.5, color='green')
plt.title('Budget vs Revenue for other_movies(95%) ', fontsize=16)
plt.xlabel('Budget', fontsize=14)
plt.ylabel('Revenue', fontsize=14)
plt.show()
```



Summary

In this project, I worked with the TMDB Movies Dataset to analyze movie trends. The dataset contained information such as movie titles, budgets, revenues, genres, and ratings. Here's what I did:

Explored the Data

- Examined the dataset's structure, basic statistics, and identified missing values.

Cleaned the Data

- Removed unnecessary columns.
- Filled in missing values.
- Dropped rows with missing critical information.

Analyzed Genres

- Found that **Drama** is the most popular genre every year, followed by **Comedy** and **Thriller**.

Visualized Trends

- Created charts to display genre popularity over time.
- Analyzed the relationship between budget and revenue.

What I Learned

- **Data cleaning is crucial** before performing any analysis.
- **Visualizing data** makes it easier to understand trends.
- **Drama remains a favorite**, and **big budgets can lead to big earnings**.

Final Thoughts

This project was insightful and helped me gain valuable experience in working with real-world data.

Conclusion

After analyzing the TMDB Movies dataset, here's what I found:

Drama is King

- Drama is consistently the most popular genre year after year.
- It also has the highest number of movies produced.
- Audiences seem to love emotional, intense storytelling.

Comedy Comes Second

- Comedy is the **second most popular genre**, with over **3,700 movies** in the dataset.
- It appears that audiences enjoy humor just as much as drama.

Vote Averages

- Most movies receive **average ratings**, but there are outliers.
- Some films are **highly praised**, while others are **widely disliked**.

Limitations of the Analysis

While this analysis provided interesting insights, there were some challenges and limitations that made drawing perfect conclusions difficult:

Data Cleaning Was Tricky

- The dataset contained **a lot of missing or messy data**.
- Columns like `genres` and `vote_average` had many empty values.
- Dropping missing rows might have resulted in **loss of useful information**.

Genre Splitting Was Complicated

- The `genres` column had **multiple genres listed together** (e.g., `Action|Adventure|Sci-Fi`).
- Splitting them into separate rows required **trial and error**.
- There was a chance of **incorrect splits** affecting analysis accuracy.

Outliers in Vote Averages

- The `vote_average` column had **extremely high or low values**.
- It was difficult to determine if these were **errors or just unpopular/overhyped movies**.
- Keeping them **may have skewed** some results.

Hard to Visualize Trends

- Plotting **genre popularity over time** resulted in **overcrowded graphs**.
- The sheer number of genres made it **difficult to create clear and readable charts**.
- Simplifying the data meant **losing some details**.

Limited by My Coding Skills

- As a **beginner in Python and Pandas**, some tasks were challenging.
- **Splitting, exploding, and grouping data** by year and genre took a lot of research and experimentation.
- There are probably **better, more efficient methods**, but this was the best I could achieve for now. 😊 e up with for now.